FAILURE ANALYSIS & SELF-HEALING

R26-SE-039

Project Proposal Report

Ranasinghe S.I – IT22371768

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Ranasinghe S.I	IT22371768	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mrs. Suriya Kumari	14/03/2026	

ABSTRACT

Software testing in modern CI/CD environments often suffers from unstable test cases, flaky failures, and long debugging times, which reduce the efficiency of continuous integration pipelines. Existing automated testing frameworks can detect failures but they do not effectively identify the root causes or automatically repair failing tests. This research proposes an intelligent automated testing framework that integrates machine learning–based root cause analysis with a self-healing mechanism to improve the reliability of automated tests. The system collects test execution artifacts such as logs, screenshots, stack traces, and test metadata from CI/CD pipelines including Jenkins, GitHub Actions, and GitLab CI/CD. Machine learning models such as Random Forest and Gradient Boosting are used to classify failure causes and determine whether failures can be automatically repaired or should be reported to developers. A self-healing engine attempts to repair broken locators and unstable test scripts, while predictive analytics identifies flaky and unstable tests using historical execution data. The proposed system is expected to reduce manual debugging effort, improve test stability, and enhance CI/CD pipeline reliability. From a social and commercial perspective, the solution can help software organizations reduce testing costs, accelerate software delivery, and improve product quality.

Keywords: Automated Testing, Self-Healing Tests, Machine Learning, CI/CD Pipelines, Root Cause Analysis

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor and lecturers at the Sri Lanka Institute of Information Technology for their valuable guidance and support during the preparation of this research proposal. Their insights and recommendations have helped shape the direction of this research.

I would also like to thank my project team members for their cooperation and collaboration throughout the project development process. Finally, I would like to acknowledge the academic resources and facilities provided by SLIIT which enabled the successful preparation of this proposal.

TABLE OF CONTENTS

DECLARATION 2
ABSTRACT 3
ACKNOWLEDGEMENT 4
LIST OF TABLES 7
LIST OF FIGURES 8
LIST OF ABBREVIATIONS 9
LIST OF APPENDICES 10
1. INTRODUCTION 11
1.1 Background and Context 11
1.2 Literature Review 12
1.3 Research Gap 13
1.3.1. Research Value Aspect 13
1.3.2. Product Value Aspect 13
1.3.3. Evidence-Based Comparison 14
2. OBJECTIVE 15
2.1 Main Objective 15
2.2 Specific Objectives 15
3. METHODOLOGY 16
3.1 Research Approach 16
3.2 Technical Methods and Algorithms 16
3.3 Tools and Platforms with Justification 18
3.4 Data Collection and Ethical Considerations 22
3.5 Validation Strategy and Performance Metrics 24
4. HIGH-LEVEL SYSTEM ARCHITECTURE 25
4.1 Overview of the Proposed Solution and Its Components 25
4.2 System Diagram of Components 25
4.3 Individual Component and Its Parts 26
4.4 Component Interactions 27
4.5 Data Flow Explanation 27
4.6 Scalability and Security Considerations 28
5. USER REQUIREMENTS 28
5.1 Stakeholders 28

5.2 Functional Requirements 29
5.3 Non Functional Requirements 30
6. COMMERCIALIZATION PLAN 31
6.1. Target Market and Customer Persona 31
6.2. Value Proposition 32
6.3. Revenue Model 32
6.4. Cost Estimation 32
6.5. Pricing Strategy 33
6.6. Competitive Advantage 33
6.7. Intellectual Property 34
7. BUDGET AND JUSTIFICATION 35
7.1 Hardware Costs 35
7.2 Software Licenses 35
7.3 Cloud Services 35
7.4 Human Effort Cost 36
7.5 Total Estimated Budget 36
8. WORK BREAKDOWN STRUCTURE (WBS) 37
9. GANTT CHART 38
10. REFERENCES LIST 39
11. APPENDICES 41

LIST OF TABLES

Table 1: Evidence-Based Comparison..... 14
Table 2: Tools and Platforms Used for System Implementation 21
Table 3: Stakeholder Analysis..... 29
Table 4: Cost Estimation for System Development..... 33
Table 5: Pricing Strategy and Subscription Plans 33
Table 6: Budget Allocation for Hardware 35
Table 7: Budget Allocation for Software..... 35
Table 8: Budget Allocation for Cloud Services..... 35
Table 9: Budget Allocation for Human Effort 36
Table 10: Total Estimation Budget..... 36
Table 11: Risk Analysis..... 43

LIST OF FIGURES

Figure 1: High-Level Architecture of the NEXTGEN QA System 26
Figure 2: System Architecture Diagram for Individual Component 26
Figure 3: Work Breakdown Structure Diagram 37
Figure 4: Gantt Chart 38
Figure 5: Data flow diagram for failure analysis & self-healing 42
Figure 6: UI of Failure analysis dashboard 42
Figure 7: UI of Self-healing test recommendation panel 43
Figure 8: Turnitin Report 44

LIST OF ABBREVIATIONS

Abbreviation	Description
RP	Research Project
CI/CD	Continuous Integration and Continuous Deployment
ML	Machine Learning
RF	Random Forest
XGBoost	Extreme Gradient Boosting
DOM	Document Object Model
UI	User Interface
DSR	Design Science Research
RTM	Requirement Traceability Matrix

LIST OF APPENDICES

Appendix A: Survey Instruments 41
Appendix B: Detailed System Diagrams 41
Appendix C: Preliminary UI Mockups 42
Appendix D: Risk Analysis 43
Appendix E: Ethical Considerations 43
Appendix F: Originality Report 43

1. INTRODUCTION

1.1 Background and Context

Modern software development practices rely heavily on Continuous Integration and Continuous Deployment (CI/CD) pipelines to ensure fast and reliable software delivery. Automated testing frameworks such as Selenium, Cypress, and Playwright are widely used in these pipelines to detect software defects during development. Automated tests are executed continuously to verify that new code changes do not break existing system functionality. However, maintaining reliable automated test suites has become a significant challenge for software development teams. Automated tests frequently fail due to various reasons such as user interface changes, unstable element locators, synchronization issues, environment configuration problems, network instability, and inconsistent test data. These failures often require QA engineers to manually inspect logs, stack traces, and execution artifacts in order to identify the root cause of the problem. Studies have shown that a large percentage of automated test failures are not caused by real software defects but by unstable test scripts and environmental issues. This situation leads to increased debugging time, reduced efficiency of CI/CD pipelines, and higher maintenance costs for automated test suites.

To address these challenges, the NEXTGEN QA framework proposes an intelligent automated testing system consisting of four main components: Voice/NLP Intelligence, Test Generation and Execution Intelligence, Failure Analysis and Self-Healing Intelligence, and Quality Prediction with Requirement Traceability Matrix (RTM) generation. Within this framework, the Failure Analysis and Self-Healing component focuses on automatically analyzing test failures, identifying their root causes, and repairing unstable automated tests. Several existing automated testing tools such as Testim, Katalon Studio, and Functionize provide basic self-healing capabilities, particularly for repairing broken UI locators. However, most of these tools lack intelligent mechanisms for analyzing the root causes of failures or predicting unstable tests before they fail. Therefore, there is a need for a more advanced solution that combines machine learning-based failure analysis, predictive alerts, and automated test repair mechanisms without human intervention.

This research aligns with **Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure)** because it supports the development of more efficient and reliable software infrastructure. It also aligns with the **Software Systems and Technologies research cluster**, which focuses on intelligent systems and software automation.

1.2 Literature Review

Automated testing is an essential component of modern software engineering, yet maintaining reliable automated test suites remains a significant challenge. Berner et al. investigated the maintenance challenges associated with automated testing and emphasized the high cost of diagnosing test failures in large systems [1].

Yoo and Harman conducted a comprehensive survey of regression testing techniques, including test case prioritization and minimization strategies aimed at improving testing efficiency while maintaining adequate coverage [2].

Data-driven approaches have also been explored to improve software testing processes. Feldt et al. demonstrated that analyzing test execution behavior can improve test quality and provide insights into system reliability [3].

Flaky tests represent another major challenge in automated testing environments. Luo et al. performed a systematic literature review on flaky test detection techniques and demonstrated that analyzing historical test execution data can improve failure detection accuracy [4].

GUI-based testing approaches have also been studied extensively. Memon proposed an event-flow model for GUI testing which explains how UI changes can cause automated test failures due to locator inconsistencies [5].

Research on test suite effectiveness has also explored mutation testing and test case prioritization techniques. Jia and Harman analyzed mutation testing as a technique to evaluate the fault detection capability of test suites [6]. Noor and Hemmati reviewed various test case prioritization strategies designed to detect faults earlier during testing [7].

Industry reports also indicate a growing adoption of AI-driven testing tools. The BrowserStack State of Software Testing Report highlights the increasing reliance on automated testing frameworks integrated with CI/CD pipelines [8]. Commercial tools such as Testim and Functionize provide basic AI-assisted self-healing features.

However, most existing tools focus primarily on locator healing and do not provide deep root cause analysis or predictive failure detection. This limitation highlights the need for more intelligent automated testing frameworks.

1.3 Research Gap

1.3.1. Research Value Aspect

Existing research in automated testing mainly focuses on improving test execution efficiency, regression testing strategies, and defect detection techniques. However, diagnosing the root causes of automated test failures still requires significant manual effort. Studies highlight that automated test maintenance and failure debugging remain costly and time-consuming tasks in large systems **[1]**. Research on regression testing and test execution analysis improves testing efficiency but does not address intelligent failure diagnosis **[2][3]**. Additionally, flaky test detection techniques help identify unstable tests but do not provide mechanisms to automatically repair failing tests **[4]**. Therefore, there is a research gap in developing an integrated framework that can analyze failure artifacts, classify root causes using machine learning, and support automated repair of unstable tests.

.

1.3.2. Product Value Aspect

Several existing tools support automated testing and CI/CD integration, including **Selenium, Testim, Mabl, and Functionize**. These tools provide strong capabilities for automated test execution and test maintenance. However, many of these systems still rely heavily on manual intervention when failures occur. For example, Selenium is widely used for UI test automation but it does not provide built-in mechanisms for **automatic root cause analysis or intelligent test repair**. Similarly, AI-based testing tools such as Testim and Mabl offer limited self-healing capabilities mainly focused on **UI locator updates**, but they do not provide comprehensive **machine learning-based failure classification or predictive analysis for unstable tests**.

These limitations indicate that existing products lack an integrated system capable of analyzing test artifacts, identifying failure causes, repairing tests automatically, and predicting future instability within CI/CD pipelines**[8].** The proposed system aims to address these limitations by combining machine learning models for root cause classification with a self-healing engine and predictive analytics module. This approach introduces a novel capability where the system not only detects failures but also **learns from historical execution data and automatically improves test reliability over time**, thereby reducing manual maintenance effort and improving the stability of automated testing pipelines.

1.3.3. Evidence-Based Comparison

Feature	**Selenium**	**Testim**	**Mabl**	**Proposed System**
Automated Test Execution	Yes	Yes	Yes	Yes
CI/CD Integration	Yes	Yes	Yes	Yes
Root Cause Failure Classification	No	Limited	Limited	Yes (ML based)
Self-Healing Test Scripts	No	Partial	Partial	Yes
Flaky Test Prediction	No	Limited	Limited	Yes
Predictive Test Analytics	No	No	Limited	Yes

Table 1: Evidence-Based Comparison

2. OBJECTIVE

2.1 Main Objective

To design and develop an intelligent machine learning based framework that automatically analyzes automated test failures, identifies root causes, and performs self-healing of broken test scripts in CI/CD pipelines.

2.2 Specific Objectives

1. To collect and analyze automated test execution artifacts such as logs, stack traces, screenshots, and execution metadata from CI/CD pipelines.
2. To develop a machine learning model capable of classifying root causes of automated test failures including locator issues, synchronization problems, environment failures, and application defects.
3. To design and implement a self-healing mechanism that automatically repairs unstable test scripts by updating UI locators and synchronization strategies.
4. To develop a predictive analysis module that detects flaky tests using historical test execution data.
5. To implement an adaptive test evolution mechanism that improves automated test stability based on failure patterns and repair outcomes.
6. To evaluate the performance of the proposed system using metrics such as classification accuracy, healing success rate, and reduction of debugging time.

3. METHODOLOGY

3.1 Research Approach

- This research adopts a **Design Science Research (DSR) approach** to design and evaluate an intelligent framework for failure analysis and self-healing in automated testing environments. Design Science Research is appropriate because the objective of this study is to create a technological artifact that solves a real-world problem faced in modern software development environments.

- The proposed solution addresses the instability of automated test suites used in Continuous Integration and Continuous Deployment (CI/CD) pipelines. The system will analyze test execution failures, classify their root causes using machine learning techniques, and automatically repair unstable test scripts. In addition, the system will detect flaky tests and improve test stability over time using adaptive learning mechanisms.

- The research process consists of the following stages:
  1. Collection of automated test execution artifacts
  2. Feature extraction and preprocessing
  3. Root cause classification using machine learning models
  4. Decision analysis and automated repair generation
  5. Predictive detection of flaky tests
  6. Adaptive improvement of automated test scripts
  7. Experimental evaluation of system performance

3.2 Technical Methods and Algorithms

The proposed framework consists of several modules that operate sequentially to analyze test failures and improve automated testing reliability.

Failure Artifact Collection

- Automated test execution artifacts will be collected from CI/CD pipelines. These artifacts include execution logs, stack traces, error messages, screenshots, HTTP response codes, and environment metadata. These data sources provide valuable information about the causes of test failures.

Feature Extraction

- The collected artifacts will be processed to extract relevant features for machine learning analysis. Natural Language Processing techniques such as tokenization and TF-IDF vectorization will be applied to analyze textual log data. Structured features such as exception types, execution duration, retry counts, browser type, and response codes will also be extracted.

Root Cause Classification

- A supervised machine learning model will be developed to classify the root causes of automated test failures. Algorithms such as **Random Forest and Gradient Boosting** will be trained using labeled datasets of historical test failures. The model will categorize failures into classes such as locator issues, synchronization issues, environment failures, network or API errors, test data issues, and application defects.

Decision Analysis

- After the failure cause is identified, a decision module determines the appropriate action. For example, locator failures will trigger locator repair, synchronization failures will trigger adjustments to waiting strategies, and network failures may trigger test retries. If the failure is caused by an application defect, the system will report the issue without performing automated repair.

Self-Healing Mechanism

- The self-healing module automatically modifies test scripts to repair unstable tests. For example, if a user interface element locator changes, the system analyzes the current DOM structure to identify alternative element attributes and updates the test script accordingly. The repaired test is then re-executed to verify the correction.

Predictive Failure Detection

- To detect unstable tests, historical test execution data will be analyzed to identify flaky tests. Machine learning techniques will detect patterns of inconsistent pass and fail results across multiple test runs.

Adaptive Test Evolution

- The system stores historical failure data and repair outcomes to continuously improve automated test stability. By learning from previous failures, the system gradually improves locator strategies and test execution reliability.

3.3 Tools and Platforms with Justification

Several technologies will be used to implement the proposed framework.

- **Python** will be used as the primary programming language because it provides extensive libraries for data processing and machine learning development.
- **Scikit-learn** will be used to implement machine learning models for root cause classification and predictive analysis.
- **Selenium** will be used as the automated testing framework because it is widely used for web application testing and generates detailed execution artifacts such as logs and screenshots.
- **BeautifulSoup or DOM parsing libraries** will be used to analyze web page structures and identify alternative UI locators for the self-healing mechanism.
- **Jenkins or GitHub Actions** will be used to simulate CI/CD pipelines and execute automated test cases.
- **MySQL or MongoDB** will be used to store historical test execution data, failure logs, and repair outcomes required for predictive analysis and adaptive learning.

Tool / Platform	Purpose	Alternative Tools	Justification for Selection
Python	Core development language for implementing machine learning models and data processing.	Java, R	Python was selected because it has stronger machine learning ecosystem support such as Scikit-learn, Pandas, and NumPy. Compared to Java and R, Python offers better integration with AI frameworks and faster

			development for research prototypes.
Scikit-learn	Implementation of machine learning models for root cause classification.	TensorFlow, PyTorch	Scikit-learn is more suitable for classical machine learning algorithms such as Random Forest and Gradient Boosting. TensorFlow and PyTorch are mainly designed for deep learning and introduce unnecessary complexity for the classification tasks used in this research.
Pandas & NumPy	Data preprocessing and feature extraction from test logs and execution data.	Apache Spark, MATLAB	Pandas and NumPy provide efficient data manipulation capabilities and are widely used in machine learning pipelines. Compared to Spark, they are simpler to use for medium-scale datasets and integrate seamlessly with Python ML libraries.
Selenium	Automated UI testing and test execution for web applications.	Cypress, Playwright	Selenium was selected because it supports multiple programming languages, browsers, and CI/CD integrations. Compared with Cypress, Selenium has broader industry adoption and better compatibility with existing automation frameworks.
JUnit / TestNG	Automated test execution framework.	NUnit, PyTest	JUnit and TestNG are widely used in enterprise testing environments and produce structured execution reports that are useful for failure

			analysis. These frameworks also integrate easily with CI/CD tools such as Jenkins.
Jenkins	CI/CD pipeline automation and execution of automated tests.	GitHub Actions, GitLab CI	Jenkins was selected due to its extensive plugin ecosystem and flexibility in configuring complex pipelines. It is also widely used in industry environments for continuous testing and integration.
MySQL / PostgreSQL	Storage of test execution data, logs, and failure classification results.	MongoDB, SQLite	Relational databases such as MySQL and PostgreSQL provide structured data storage and efficient querying capabilities. Compared with NoSQL databases like MongoDB, relational databases are better suited for structured testing datasets and analytical queries.
NLTK / spaCy	Natural language processing of error logs and failure messages.	Stanford NLP, TextBlob	spaCy and NLTK provide lightweight NLP capabilities that are sufficient for log message analysis. Compared to larger frameworks like Stanford NLP, they require fewer computational resources and are easier to integrate into Python-based pipelines.
Docker	Containerization of the experimental environment.	Virtual Machines, Kubernetes	Docker was selected because it provides lightweight containerization and ensures

			reproducibility of the research environment. Compared with virtual machines, Docker requires fewer resources and allows faster deployment.
Git / GitHub	Version control and collaboration for project development.	SVN, Bitbucket	Git is the most widely adopted distributed version control system. GitHub provides integrated collaboration features, issue tracking, and version history management that support research development workflows.

Table 2: Tools and Platforms Used for System Implementation

These tools were selected because they are widely used in modern DevOps environments and support integration with automated testing pipelines.

3.4 Data Collection and Ethical Considerations

The dataset used in this research will be collected from **simulated CI/CD testing environments** and **open-source web application testing projects**. Since the proposed component focuses on failure analysis and self-healing, the data source will primarily consist of automated test execution artifacts generated when web application test scripts are executed repeatedly under controlled testing conditions.

Data Sources

- The data will be collected from the following sources:
  1. **Simulated CI/CD Pipelines**
     Automated test cases will be executed using tools such as Selenium integrated with Jenkins or GitHub Actions. These executions will generate realistic failure artifacts under controlled conditions.
  2. **Open-Source Demo Web Applications**
     Publicly available testable web applications such as e-commerce, login, and booking systems will be used to generate automated test cases and intentional failure scenarios.
  3. **Locally Created Test Scenarios**
     Additional unstable test cases will be created intentionally by modifying UI locators, synchronization timing, test data, and environment configurations to produce different categories of failures for model training.

Types of Data Collected

The following data types will be collected for each automated test execution:

- execution logs
- stack traces
- exception types
- screenshots captured during failures
- DOM snapshots or HTML structure
- HTTP response codes
- execution time
- retry count
- browser and environment metadata

- pass/fail status
- manually assigned root cause label

These artifacts will provide both **textual** and **structured** data required for machine learning-based root cause classification and predictive analysis.

Expected Dataset Size

The study aims to collect approximately **1,000 to 3,000 automated test execution instances** during the experimental phase. Among these, a substantial subset will be **failure cases**, intentionally generated across multiple categories such as:

- locator issues
- synchronization issues
- test data issues
- network or API failures
- environment failures
- application defects

- A balanced labeled dataset of approximately **500 to 1,000 failure instances** will be prepared for machine learning model training and validation.

Data Labeling

To train the root cause classification model, failed test executions will be **manually reviewed and labeled** according to their actual causes. Each failure instance will be assigned to one of the predefined categories based on evidence from logs, stack traces, screenshots, and execution metadata. This labeled dataset will then be used to train supervised machine learning models.

Ethical Considerations

This research primarily uses **technical system-generated testing data** rather than personal user information. Therefore, the risk of privacy violations is minimal. However, any sensitive project-specific details or confidential system identifiers appearing in logs or screenshots will be anonymized before analysis.

The collected data will be used strictly for academic research purposes and stored securely. No personal user data, credentials, or private business information will be intentionally collected as part of this research.

3.5 Validation Strategy and Performance Metrics

- The performance of the proposed system will be evaluated through experimental testing using automated test execution datasets.
- The root cause classification model will be evaluated using standard machine learning metrics including **accuracy, precision, recall, and F1-score**.
- The effectiveness of the self-healing mechanism will be evaluated using the **healing success rate**, which measures the percentage of failed tests successfully repaired by the system.
- The predictive module will be evaluated based on its ability to correctly identify flaky tests using historical execution data.
- Additionally, the overall effectiveness of the framework will be measured by the **reduction in manual debugging time and improvement in automated test stability within CI/CD pipelines**.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed Solution and Its Components

The proposed system, **NEXTGEN QA**, is an intelligent automated testing framework designed to improve the reliability and efficiency of CI/CD pipelines. The system integrates multiple intelligent modules that support requirement extraction, automated test generation, failure analysis, and quality prediction.

The architecture of the proposed solution consists of **four main components**.

The first component is **Voice/NLP Intelligence**, which processes voice discussions and meeting transcripts to extract structured software requirements using speech-to-text and natural language processing techniques.

The second component is **Test Generation and Execution Intelligence**, which converts the extracted requirements into automated test cases and test scripts. These tests are executed in CI/CD environments using automated testing frameworks.

The third component is **Failure Analysis and Self-Healing Intelligence**, which focuses on analyzing test execution failures. This component identifies the root causes of test failures, generates automated repair strategies, predicts unstable tests, and continuously improves test stability. This component represents the main focus of this research.

The fourth component is **Quality Prediction and RTM Generation**, which analyzes testing results and generates a Requirement Traceability Matrix (RTM) and software quality insights for project stakeholders.

Together, these components create an intelligent end-to-end testing ecosystem that reduces manual effort and improves automated testing reliability.

4.2 System Diagram of Components

The high-level system architecture consists of four interconnected components that exchange data throughout the testing lifecycle.

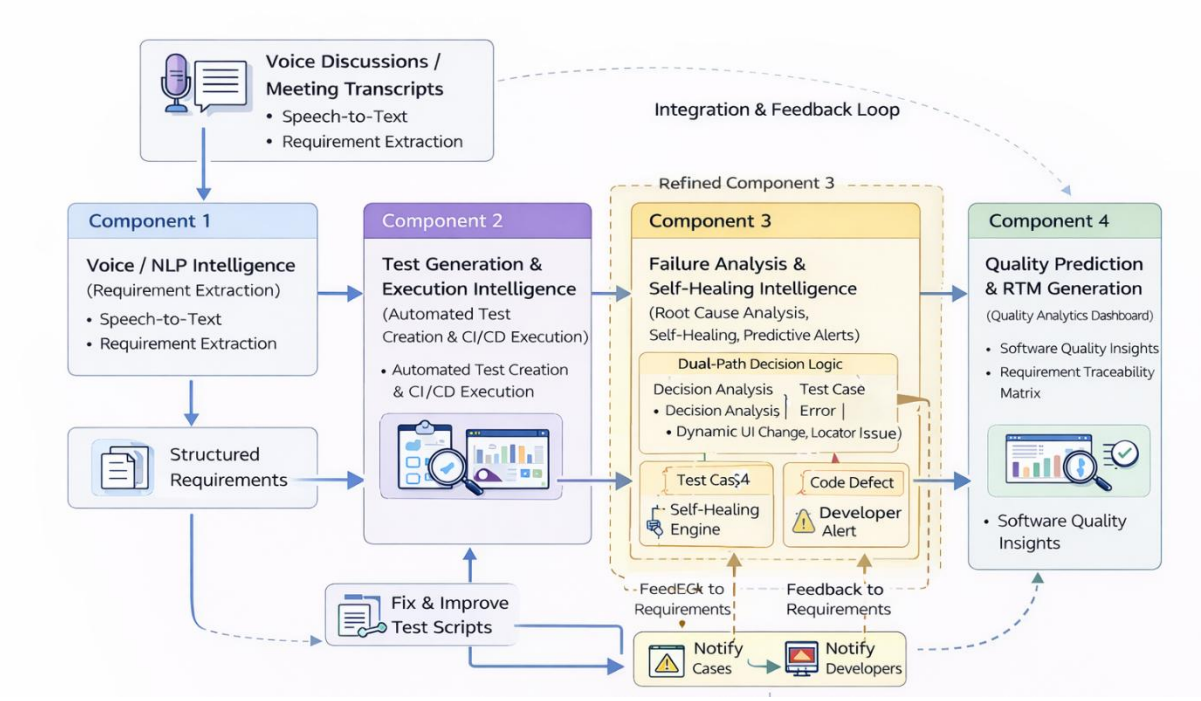


Figure 1: High-Level Architecture of the NEXTGEN QA System

4.3 Individual Component and Its Parts

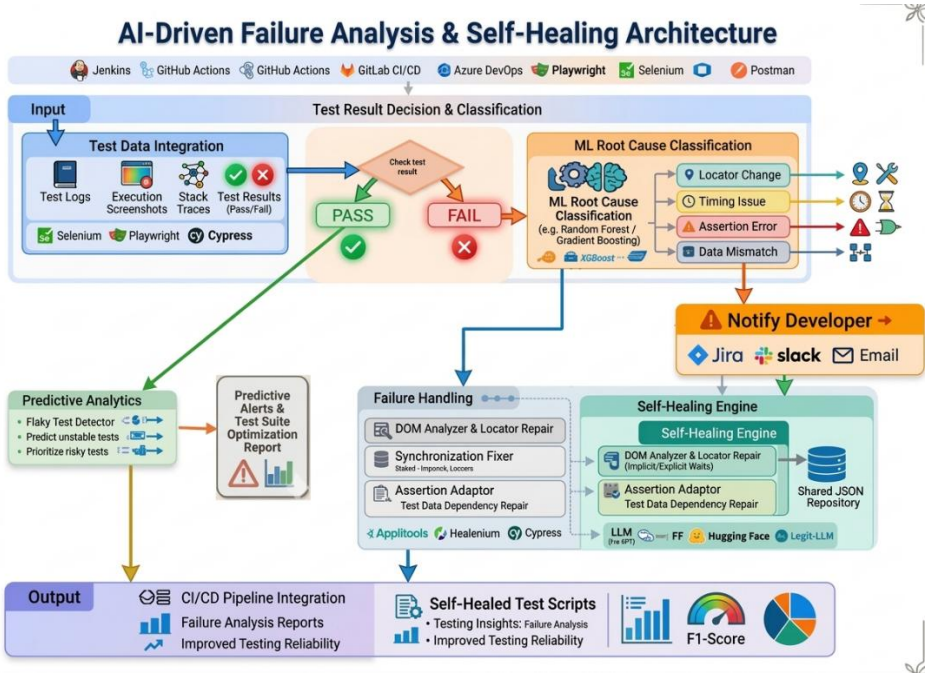


Figure 2: System Architecture Diagram for Individual Component

The **Failure Analysis and Self-Healing Intelligence component** consists of several submodules that analyze automated test failures and improve test reliability.

These modules include:

• **Failure Artifact Collection Module** – collects logs, stack traces, screenshots, and execution metadata from CI/CD pipelines.

• **Feature Extraction Module** – processes failure artifacts and extracts meaningful features such as exception types, execution time, and log patterns.

• **Root Cause Classification Module** – applies machine learning algorithms to classify failure causes including locator issues, synchronization problems, environment failures, and application defects.

• **Decision Analysis Module** – determines the appropriate response based on the predicted failure cause.

• **Self-Healing Engine** – automatically repairs unstable test scripts by updating UI locators or synchronization strategies.

• **Predictive Failure Detection Module** – identifies flaky tests by analyzing historical execution data.

• **Adaptive Test Evolution Module** – continuously improves automated test stability by learning from past failures and repair outcomes.

4.4 Component Interactions

Each component of the NEXTGEN QA system interacts with others to create a complete automated testing ecosystem.

Component 1 extracts structured requirements from voice discussions and passes them to Component 2.

Component 2 generates automated test cases and executes them within CI/CD pipelines.

The test execution results, logs, and artifacts are passed to Component 3 for failure analysis and automated repair.

Component 3 produces reliability insights and improved test scripts, which are then used by Component 4 to generate software quality predictions and requirement traceability matrices.

4.5 Data Flow Explanation

The system follows a sequential data flow starting from requirement extraction to quality prediction.

First, voice inputs are converted into structured requirements using NLP techniques. These requirements are then used to automatically generate test cases and scripts.

When tests are executed in the CI/CD environment, execution artifacts such as logs and screenshots are generated. These artifacts are analyzed by the failure analysis module to determine the root causes of failures.

Based on the identified cause, automated repair strategies are applied to stabilize the test scripts. Historical testing data is stored to enable predictive analysis of flaky tests and adaptive improvement of test stability.

4.6 Scalability and Security Considerations

The proposed system is designed to support large-scale CI/CD environments where thousands of automated tests may be executed daily.

To ensure scalability, the architecture supports modular integration with CI/CD tools such as Jenkins and GitHub Actions. Machine learning models can also be retrained periodically using updated testing data.

From a security perspective, the system ensures that sensitive system logs and testing artifacts are securely stored and access-controlled. Any confidential project data used in the research will be anonymized to protect privacy and comply with ethical research standards.

5. USER REQUIREMENTS

5.1 Stakeholders

The proposed intelligent automated testing framework will involve several stakeholders who interact with different parts of the system.

Stakeholder	Description
QA Engineers / Testers	Use the system to monitor automated test executions, analyze failures, and manage repaired test scripts.
Software Developers	Receive notifications when failures are caused by application defects requiring code fixes.
DevOps Engineers	Integrate the framework into CI/CD pipelines and manage automated test executions.
Project Managers	Monitor software quality metrics and testing progress through the quality analytics dashboard.
Research System (Automated Agent)	The intelligent module that performs failure analysis, classification, and self-healing operations automatically.

Table 3: Stakeholder Analysis

5.2 Functional Requirements

The system must support the following functional capabilities.

FR1: The system shall collect automated test execution artifacts such as logs, stack traces, screenshots, and execution metadata from CI/CD pipelines.

FR2: The system shall analyze failure artifacts and extract relevant features required for root cause analysis.

FR3: The system shall classify automated test failures into predefined categories such as locator issues, synchronization errors, environment failures, and application defects.

FR4: The system shall perform decision analysis to determine whether a failure is caused by a test script issue or a software application defect.

FR5: The system shall automatically repair unstable test scripts when failures are caused by test case issues such as dynamic UI changes or incorrect locators.

FR6: The system shall generate notifications for software developers when failures are identified as application-level defects.

FR7: The system shall maintain a historical dataset of test executions and failure cases to support predictive analysis.

FR8: The system shall identify flaky or unstable tests by analyzing historical execution patterns.

FR9: The system shall update repaired test scripts and store the improved scripts for future executions.

FR10: The system shall provide failure analysis insights to the quality prediction module to support overall software quality evaluation.

5.3 Non Functional Requirements

Non-functional requirements define system quality attributes such as performance, reliability, and security.

Performance: The system should analyze test failures and generate classification results within a reasonable time after test execution.

Scalability: The system should support large-scale automated testing environments where hundreds or thousands of test cases are executed in CI/CD pipelines.

Reliability: The system should maintain accurate failure classification and minimize incorrect automated repairs.

Security: All testing artifacts and execution logs should be securely stored, and any sensitive project data should be anonymized.

Usability: The system should provide clear notifications and failure analysis results to testers and developers.

Maintainability: The architecture should allow easy integration of additional machine learning models and testing frameworks.

6. COMMERCIALIZATION PLAN

6.1. Target Market and Customer Persona

The proposed system targets software development companies that rely heavily on automated testing and CI/CD pipelines. These organizations typically follow Agile and DevOps development practices and require reliable automated testing frameworks to maintain software quality while delivering frequent software updates.

Potential customers include:

- Software development companies
- Quality assurance teams
- DevOps teams
- Technology startups
- Enterprises maintaining large automated test suites

A typical customer persona for the proposed solution is a QA Automation Engineer or DevOps Engineer responsible for maintaining automated test pipelines. These professionals face challenges such as flaky tests, frequent test failures, and long debugging times. The proposed system helps these users reduce debugging effort and improve the reliability of automated testing.

6.2. Value Proposition

- The proposed system provides significant value to software development teams by improving the reliability of automated testing environments.
- The key value propositions include:
  - Automatic identification of root causes of test failures
  - Reduction of manual debugging effort for QA teams
  - Automated repair of unstable test scripts
  - Early detection of flaky or unstable tests
  - Faster CI/CD pipelines due to reduced test failures
- By automating failure diagnosis and repair, the system helps organizations **save time, reduce maintenance costs, and improve software quality**.

6.3. Revenue Model

The proposed system can be commercialized as a **cloud-based Software-as-a-Service (SaaS) platform** or as a **CI/CD plugin** integrated with existing DevOps tools.
Possible revenue models include:

- **Subscription-based licensing** for organizations using the platform
- **Enterprise licensing** for large companies
- **Plugin-based integration services** for CI/CD tools such as Jenkins or GitHub Actions

Organizations may subscribe to the platform based on the number of automated tests executed or the size of the development team.

6.4. Cost Estimation

The initial development and deployment cost of the proposed system is estimated to be approximately **$50,000** for building a functional prototype and launching the platform for early adoption.

The estimated budget allocation is as follows:

Cost Category	**Estimated Cost**
Cloud infrastructure and hosting services	12,000
Software development and system implementation	20,000

Machine learning model development and testing	8,000
Maintenance and technical support	6,000
Deployment, integration, and operational expenses	4,000

Table 4: Cost Estimation for System Development

Total Estimated Cost: **50,000**

This budget covers the development of the prototype system, deployment on cloud infrastructure, machine learning model implementation, and initial operational support.

The use of open-source technologies such as Python, Scikit-learn, Selenium, and Docker helps significantly reduce development costs while enabling scalable system deployment.

6.5. Pricing Strategy

The system will follow a **subscription-based pricing strategy** to generate sustainable revenue while remaining affordable for small and medium software development teams.

Example pricing structure:

Plan	**Monthly Price**	**Target Users**
Basic Plan	$15–$25 / month	Small development teams
Professional Plan	$50–$80 / month	Medium-sized DevOps teams
Enterprise Plan	Custom pricing	Large organizations with advanced integration

Table 5: Pricing Strategy and Subscription Plans

With this pricing model, the system can recover the initial investment cost while supporting long-term platform development and maintenance.

6.6. Competitive Advantage

Although existing automated testing tools provide test execution capabilities, most tools do not provide **intelligent failure analysis and automated self-healing features**.

The proposed system offers several competitive advantages:

- Machine learning-based root cause classification
- Automated test repair mechanisms
- Predictive detection of unstable tests
- Integration with CI/CD environments
- Continuous learning through adaptive test evolution

These capabilities provide a significant advantage over traditional test automation tools that rely heavily on manual debugging.

6.7. Intellectual Property

The proposed framework may involve intellectual property related to machine learning algorithms for failure classification and automated test repair mechanisms.

The research outputs may be protected through **software copyrights or potential patent applications** for novel failure analysis and self-healing approaches.

The system may also be released under a controlled licensing model to protect proprietary algorithms while allowing commercial deployment.

7. BUDGET AND JUSTIFICATION

The estimated budget required to design, develop, and evaluate the proposed research prototype is approximately **50,000**.

7.1 Hardware Costs

Item	Estimated Cost
High-performance workstation for development and testing	3,500
Additional testing devices and infrastructure	1,500

Table 6: Budget Allocation for Hardware

Total Hardware Cost: 5,000

Justification:

Hardware resources are required for machine learning experiments, automated testing execution, and processing of large testing datasets.

7.2 Software Licenses

Item	Estimated Cost
Development tools and IDE licenses	1,000
Testing framework plugins and integrations	1,000

Table 7: Budget Allocation for Software

Total Software Cost: 2,000

Justification:

Although most technologies used are open source, some development tools and integration plugins may require licensing.

7.3 Cloud Services

Item	Estimated Cost
Cloud servers and storage for CI/CD simulation	10,000
Database hosting and experiment environment	3,000

Table 8: Budget Allocation for Cloud Services

Total Cloud Cost: 13,000

Justification:

Cloud services are necessary to run automated testing pipelines, store datasets, and execute machine learning experiments.

7.4 Human Effort Cost

Role	Estimated Cost
Software development and machine learning implementation	20,000
System integration and testing support	10,000

Table 9: Budget Allocation for Human Effort

Total Human Effort Cost: 30,000

Justification:

Human effort represents the main cost since system design, development, machine learning model training, and testing infrastructure setup require significant technical work.

7.5 Total Estimated Budget

Category	Cost
Hardware	5,000
Software	2,000
Cloud Services	13,000
Human Effort	30,000

Table 10: Total Estimation Budget

Total Estimated Budget: 50,000

8. WORK BREAKDOWN STRUCTURE (WBS)



Figure 3: Work Breakdown Structure Diagram

9. GANTT CHART

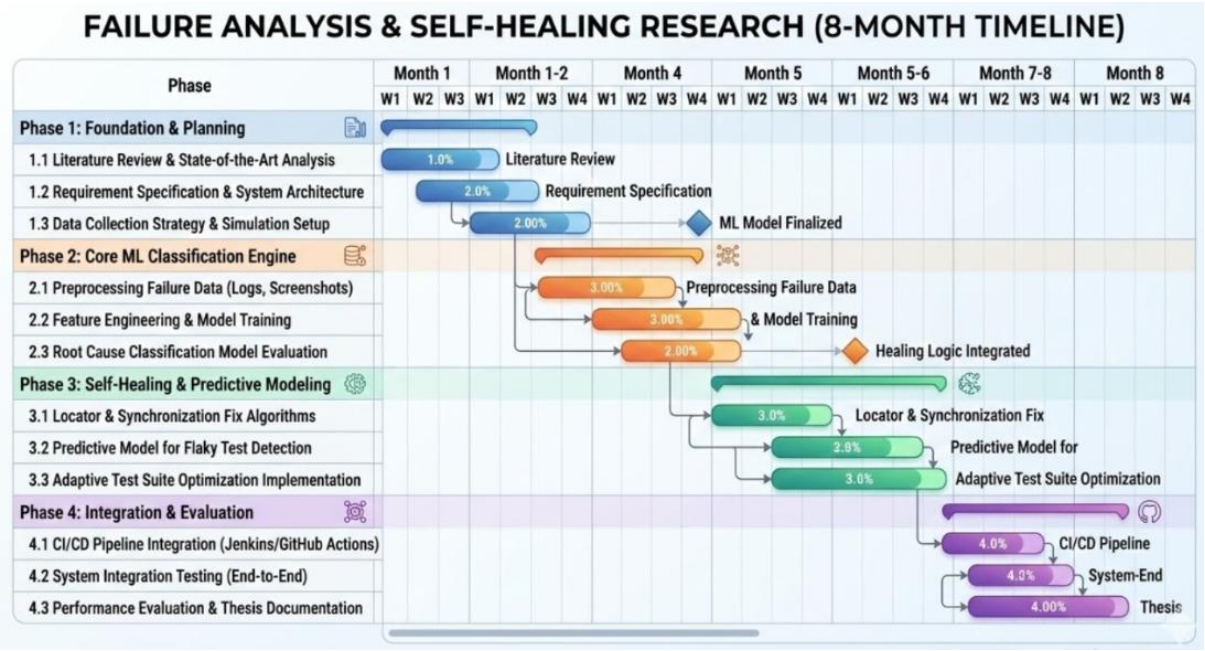


Figure 4: Gantt Chart

10. REFERENCES LIST

[1] S. W. R. Berner and K. R. K. Berner, "Observations and Lessons Learned from Automated Testing," *IEEE Computer Society Press*, Washington, DC, 2007. Available: https://doi.org/10.1109/ICSE.2007.33

[2] S. Yoo and M. Harman, "Regression Testing Minimization, Selection and Prioritization: A Survey," *John Wiley & Sons, Ltd.*, Chichester, UK, 2012. Available: https://doi.org/10.1002/stvr.365

[3] R. T. Feldt et al., "Improving Software Test Quality by Observing Test-Execution," *Association for Computing Machinery (ACM)*, New York, NY, 2018. Available: https://doi.org/10.1145/3183440.3183472

[4] Q. Luo et al., "Flaky Test Detection and Mitigation: A Systematic Literature Review," *IEEE Computer Society*, Los Alamitos, CA, 2020. Available: https://doi.org/10.1002/stvr.430

[5] A. M. Memon, "An Event-Flow Model of GUI-Based Applications for Testing," *John Wiley & Sons, Ltd.*, Chichester, UK, 2012. Available: https://doi.org/10.1007/s11219-014-9243-5

[6] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Computer Society*, Los Alamitos, CA, 2011. Available: https://doi.org/10.1016/j.jss.2019.01.009

[7] M. A. Noor and H. Hemmati, "Test Case Prioritization Techniques: A Systematic Literature Review," *Springer Science+Business Media*, New York, NY, 2015. Available: https://doi.org/10.1109/TSE.2010.62

[8] BrowserStack Inc., "The State of Software Testing 2024: Industry Report on Automation, Tools, and QA Practices," BrowserStack Inc., San Francisco, CA, 2024. Available: https://www.browserstack.com/report/state-of-testing-2024

[9] S. Sinha et al., "Automated Requirements Traceability Matrix Generation Using Information Retrieval," *Elsevier Science Publishers B.V.*, Amsterdam, Netherlands, 2019.

[10] International Organization for Standardization (ISO), "ISO/IEC 25010:2015 - Systems and Software Engineering - System and Software Quality Models," Geneva, Switzerland, 2015. Available: https://www.iso.org/standard/35733.html

11. APPENDICES

The appendices provide additional supporting materials related to the proposed intelligent automated testing framework. These materials include instruments used during the research, extended system diagrams, preliminary interface designs, risk evaluation, and ethical considerations.

Appendix A: Survey Instruments

A short survey will be conducted with **software testers and DevOps engineers** to understand common challenges in automated testing environments.

Example survey questions include:

1. How frequently do automated tests fail in your CI/CD pipeline?
2. What are the most common reasons for automated test failures?
3. How much time does your team spend diagnosing failed test cases?
4. Do you currently use any tools that automatically repair test scripts?
5. How valuable would automated failure diagnosis and self-healing tests be for your development process?

The survey responses will help validate the **real-world relevance of the proposed framework**.

Appendix B: Detailed System Diagrams

This appendix contains detailed diagrams describing the proposed intelligent testing framework, including:

- Data flow diagram for the failure analysis & Self-healing

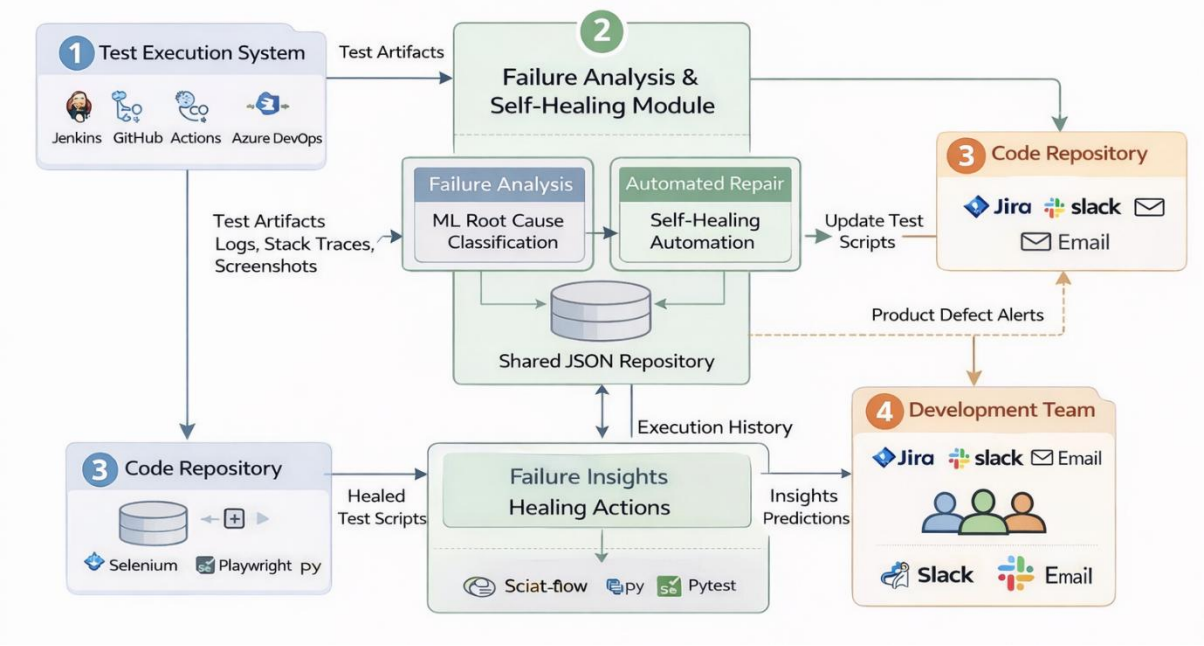


Figure 5: Data flow diagram for failure analysis & self-healing

Appendix C: Preliminary UI Mockups

Initial interface mockups for the proposed system include:

Failure analysis dashboard



Figure 6: UI of Failure analysis dashboard

Root cause classification



Figure 7: UI of Root cause classification

Self-healing test recommendation panel

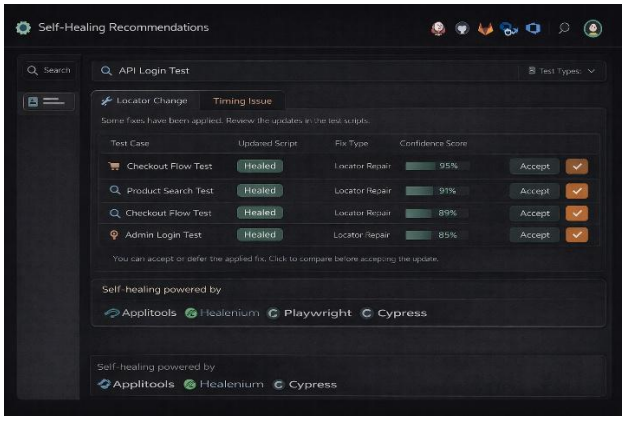


Figure 7: UI of Self-healing test recommendation panel

These mockups illustrate how developers and QA engineers will interact with the system.

Appendix D: Risk Analysis

Several risks may arise during the development and evaluation of the proposed system.

Risk	Description	Mitigation Strategy
Insufficient training data	Limited failure data for machine learning model training	Use simulated CI/CD pipelines to generate additional failure data
Model misclassification	Incorrect root cause prediction	Use cross-validation and evaluation metrics to improve accuracy
Integration challenges	Difficulty integrating with CI/CD tools	Use standardized APIs and modular architecture
System complexity	Complexity of multiple integrated components	Implement components incrementally and perform continuous testing

Table 11: Risk Analysis

Appendix E: Ethical Considerations

The research primarily analyzes **automated testing artifacts such as logs, error messages, and execution data**. No personal or sensitive user information will be collected.

All experimental datasets will be **anonymized and used solely for academic research purposes**. The research will follow standard academic guidelines for responsible data handling and ethical conduct.

Appendix F: Originality Report

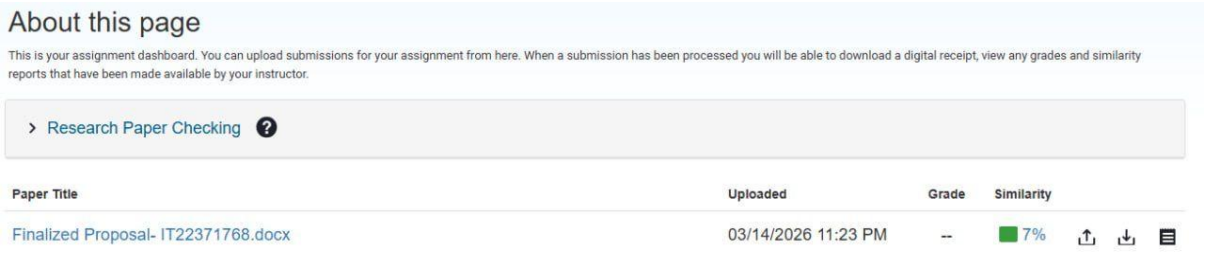


Figure 8: Turnitin Report